

Supply Chain et Sécurité

:::

Session 10 – Cours M1 Introduction aux Logiciels Libres

Objectifs de la séance

Ce que vous saurez faire

À la fin de cette séance, vous serez capables de :

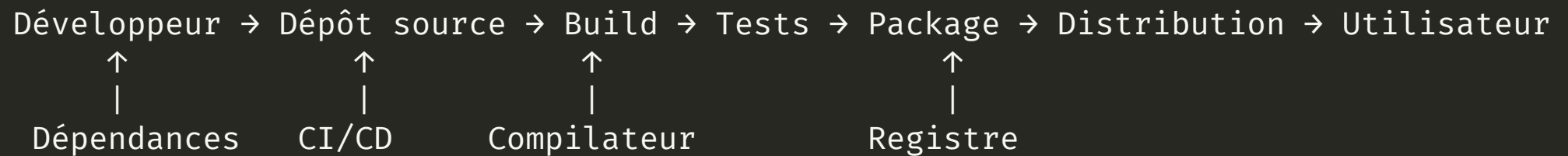
- **Comprendre** ce qu'est une supply chain logicielle
- **Identifier** les risques de sécurité spécifiques à l'open source
- **Expliquer** ce qu'est un SBOM et son importance
- **Analyser** les incidents majeurs (Log4Shell, XZ Utils)
- **Positionner** les obligations du CRA (fabricant, distributeur, steward)
- **Appréhender** l'impact des LLM sur la sécurité open source

Partie 1

La supply chain logicielle

Qu'est-ce qu'une supply chain logicielle ?

Définition : Ensemble des composants, outils, processus et personnes impliqués dans le développement et la distribution d'un logiciel.



La réalité des dépendances

Une application moderne typique :

Dépendances directes

- Bibliothèques que vous importez
- Frameworks utilisés
- Outils de build

Dépendances transitives

- Dépendances de vos dépendances
- Souvent 10x plus nombreuses
- Moins visibles, plus risquées

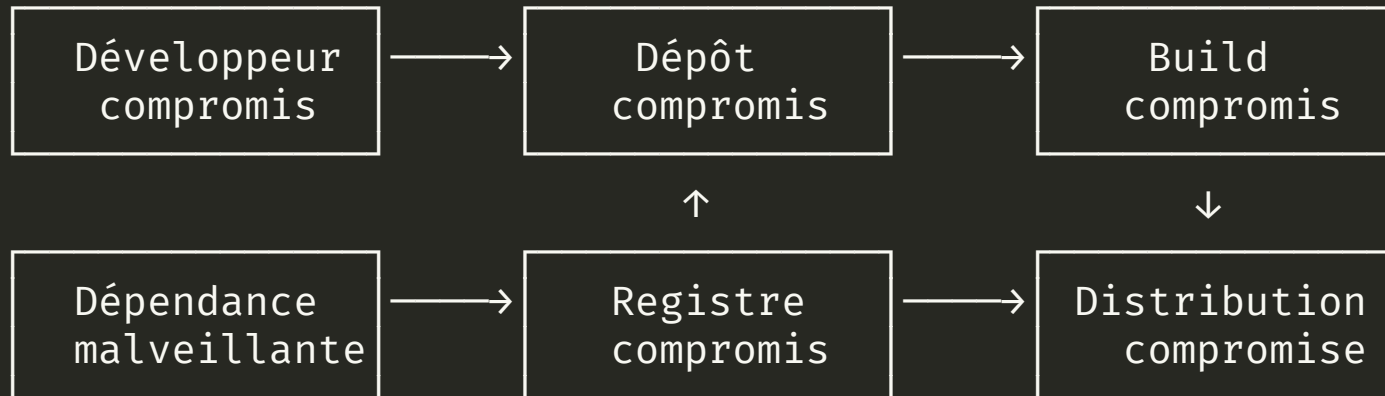
Exemple : Une app React simple peut avoir 500+ dépendances npm.

Statistiques sur les dépendances open source

Écosystème	Packages	Dépendances moyennes
npm	2+ millions	5-10 directes, 50-100 transitives
PyPI	500k+	3-5 directes, 20-50 transitives
Maven	500k+	10-20 directes, 100+ transitives
Cargo	120k+	Variable

Plus de 90% du code d'une application moderne provient de dépendances open source.

Les points d'attaque de la supply chain



Partie 2

Incidents majeurs

Log4Shell (décembre 2021)

CVE-2021-44228 – Note CVSS : **10.0** (critique)

La vulnérabilité

- Log4j : bibliothèque de logging Java
- Utilisée par des millions d'applications
- Injection JNDI permettant exécution de code

L'impact

- Exploitation triviale
- Découverte : 24 nov 2021
- Divulgation : 9 déc 2021
- Chaos mondial pendant des semaines

Log4Shell : Leçons apprises

Ce que Log4Shell a révélé ou illustré :

1. **Invisibilité des dépendances** – Beaucoup ne savaient pas qu'ils utilisaient Log4j
2. **Mainteneur unique** – Projet critique maintenu par quelques bénévoles
3. **Temps de réponse** – Difficulté à identifier et patcher tous les systèmes
4. **Dépendances transitives** – Log4j était souvent une dépendance indirecte

XZ Utils (mars 2024)

La porte dérobée la plus sophistiquée découverte

Chronologie

- 2021 : "Jia Tan" commence à contribuer
- 2022-2023 : Gagne la confiance
- 2024 : Devient co-mainteneur
- Mars 2024 : Backdoor découverte

Sophistication

- Ingénierie sociale sur 3 ans
- Code obfusqué dans les tests
- Ciblait SSH sur systèmes Debian/Fedora
- Découverte par hasard (latence)

XZ Utils : Leçons apprises

Ce que XZ a révélé :

1. **Social engineering** – L'attaquant a manipulé le mainteneur surchargé (burnout)
2. **Single maintainer** – Un seul mainteneur pour une lib critique
3. **Confiance aveugle** – Les distributions incluent automatiquement
4. **Détection difficile** – Découvert par un ingénieur Microsoft par chance

■ "We're all just one mass psychosis away from disaster."

Autres incidents notables

Incident	Année	Type	Impact
Heartbleed (OpenSSL)	2014	Vulnérabilité (buffer over-read)	Fuite de mémoire serveur, clés TLS
leftpad	2016	Suppression d'un paquet npm (11 lignes)	Des milliers de builds cassés
event-stream	2018	Mainteneur malveillant	Vol de wallets Bitcoin
ua-parser-js	2021	Compte npm compromis	Cryptominers
colors/faker	2022	Sabotage délibéré par l'auteur	Boucle infinie, protestation contre le travail non rémunéré
node-ipc	2022	Protestware (guerre Ukraine)	Fichiers écrasés sur IP russes/biélorusses
PyPI typosquatting	Continu	Faux packages	Malware divers

Partie 3

SBOM et inventaire

Qu'est-ce qu'un SBOM ?

Software Bill of Materials – Liste des ingrédients d'un logiciel.

Contient

- Tous les composants
- Leurs versions
- Leurs licences
- Leurs dépendances
- Provenance

Formats standards

- **SPDX** (Linux Foundation)
- **CycloneDX** (OWASP)
- **SWID** (ISO/IEC)

Exemple de SBOM (CycloneDX)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "components": [
    {
      "type": "library",
      "name": "log4j-core",
      "version": "2.14.1",
      "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",
      "licenses": [{"license": {"id": "Apache-2.0"}}]
    }
  ]
}
```

Pourquoi le SBOM est devenu obligatoire

Executive Order 14028 (USA, Mai 2021)

Les fournisseurs de logiciels au gouvernement fédéral doivent fournir un SBOM.

Cyber Resilience Act (UE, publié nov. 2024, applicable 11 déc. 2027)

Le CRA est le **premier règlement européen** qui encadre directement les produits numériques, y compris l'open source. 36 mois d'adaptation pour les acteurs économiques.

SBOM = Sécurité + Conformité + Soutenabilité

1. **Sécurité** – détection d'impact immédiate lors d'une nouvelle faille (type Log4Shell)
2. **Conformité CRA** – preuve de la maîtrise de la supply chain
3. **Soutenabilité** – identification des dépendances critiques à soutenir

CRA : trois rôles, trois régimes

Rôle	Qui ?	Obligations principales
Fabricant	Met un produit sur le marché (éditeur, intégrateur)	Marquage CE, SBOM, déclaration de conformité, gestion des vulnérabilités, support ≥ 5 ans
Distributeur	Revend le produit	Vérifier la conformité du fabricant, coopérer en cas d'incident
Open Source Steward	Personne morale qui soutient un projet OSS (fondation, éditeur contributeur)	Politique de cybersécurité documentée, coordination de la divulgation, signalement d'incidents graves

Nouveauté : le rôle d'**Open Source Steward** ("intendant" en Français) est créé spécifiquement par le CRA – régime allégé par rapport au fabricant.

CRA : l'exception open source

Principe : seuls les produits distribués **dans un cadre commercial** sont soumis au CRA.

Critères d'activité non commerciale

- Distribution gratuite et sans but lucratif
- Financement par dons ou subventions
- Pas de services payants associés

Un projet purement communautaire (pas d'éditeur derrière) est **hors champ** du CRA.

Stratégie de publication parallèle

L'appréciation se fait **par produit mis sur le marché** – il est possible de publier :

- Version OSS non commercialisée → *Steward*
- Version commerciale (support, SaaS, distribution payante) → *Fabricant*

Le même code, deux régimes distincts selon le produit.

Guide de conformité CNLL / inno³ : code.inno3.eu/ouvert/guide-cra

CRA : ce que ça change pour les éditeurs

Transformations internes

1. Articuler politique **open source** et politique **cyber** – convergentes mais distinctes
2. Connaître **l'intégralité** des composants intégrés (dev internes, sous-traitance, dépendances transitives)
3. Maintenir la conformité sur toute la durée de vie du produit (≥ 5 ans)

Transformations externes

- Clauses contractuelles : obtention des SBOM en amont, reversement upstream des correctifs
- Passer d'une logique d'**usage** de l'OSS à un rôle **proactif de contribution** (cf. session 9)

■ « I am not your supplier. » – **Thomas Depierre**, mainteneur OSS

Il n'y a pas de relation contractuelle entre mainteneurs et utilisateurs. Le CRA force les fabricants à l'assumer – ou à contribuer.

CRA : maturité open source

Trois niveaux de maturité des organisations face à l'open source :

Niveau	Caractéristiques
OSS subi	Usage opportuniste, informatique « grise », pas de politique
OSS maîtrisé	Risques encadrés, validation des usages, SBOM générés
OSS décidé	Engagement stratégique, contributions upstream, enjeux métiers et RH

Le CRA est un accélérateur : il force le passage du niveau 1 au niveau 2, et récompense le niveau 3 (mutualisation des coûts de cybersécurité via les communautés).

Souveraineté numérique et open source

L'open source est un levier clé de **souveraineté numérique** – la capacité à maîtriser ses infrastructures et ses données.

Risques de dépendance

- **Cloud Act** (US, 2018) : les autorités américaines peuvent exiger l'accès aux données hébergées par des entreprises US, même hors du territoire
- **Vendor lock-in** : dépendance à un éditeur unique (propriétaire ou source-available)
- Perte de contrôle sur les mises à jour, la roadmap, les conditions d'usage

L'open source comme réponse

- Code auditable et modifiable
- Pas de dépendance à un éditeur unique
- Hébergement sur infrastructure souveraine possible
- Conformité RGPD / NIS2 facilitée

Réglementation européenne

Texte	Portée	Impact open source
RGPD (2018)	Protection des données personnelles	Exige la maîtrise des données – favorise l'hébergement souverain
NIS2 (2024)	Sécurité des réseaux et systèmes d'information	Obligations de gestion des risques supply chain
CRA (2027)	Cyber-résilience des produits numériques	SBOM obligatoire, gestion des vulnérabilités
Sovereignty Package (EU, mai 2026)	Paquet de réglementations européenne annoncé prochainement	Devrait inclure une "stratégie Open Source" renforcée

Conséquence : Les organisations européennes (publiques et privées) doivent évaluer leur dépendance aux fournisseurs extra-européens et considérer des alternatives open source souveraines.

Outils de génération de SBOM

Outil	Formats	Langages
Syft (Anchore)	SPDX, CycloneDX	Multi-langage
Trivy (Aqua)	SPDX, CycloneDX	Multi-langage
cdxgen	CycloneDX	Multi-langage
pip-audit	CycloneDX	Python
uv export --format cyclonedx1.5	CycloneDX	Python
npm audit	Propriétaire	JavaScript

Partie 4

Gestion des vulnérabilités

Les bases de données de vulnérabilités

Officielles

- **CVE** (MITRE) – Identifiants
- **NVD** (NIST) – Scores CVSS
- **GHSA** (GitHub)

Spécialisées open source

- **OSV** (Google)
- **Snyk Vulnerability DB**
- **Sonatype OSS Index**

NB: dépendance vis-à-vis des US. En avril 2025, Trump & Musk (DOGE) ont coupé les budgets, notamment du MITRE. Cf.

<https://www.iisf.ie/CVE-System-grinds-to-a-near-halt>

Le score CVSS

Common Vulnerability Scoring System

Score	Sévérité	Exemple
0.0	Aucune	—
0.1 - 3.9	Faible	Info disclosure mineur
4.0 - 6.9	Moyenne	DoS partiel
7.0 - 8.9	Haute	RCE avec conditions
9.0 - 10.0	Critique	RCE sans auth (Log4Shell)

Facteurs : Vecteur d'attaque, complexité, privilèges requis, impact...

Outils de scan de vulnérabilités

SCA (Software Composition Analysis)

- **Dependabot** (GitHub)
- **Snyk**
- **OWASP Dependency-Check**
- **Trivy**
- **Grype**

Fonctionnalités

- Scan des dépendances
- Alertes automatiques
- PRs de mise à jour
- Intégration CI/CD
- Rapports de conformité

Dependabot en action

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10

  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "daily"
```

Partie 5

Bonnes pratiques

Le problème du mainteneur solitaire

"Imaginez que toute l'infrastructure numérique mondiale dépend d'un projet maintenu par un type au hasard dans le Nebraska." – Paraphrase du [XKCD #2347](#) ("Dependency")

C'est exactement ce qui s'est passé avec Log4j (2 mainteneurs bénévoles), XZ Utils (1 mainteneur burnouté), OpenSSL avant Heartbleed (1 développeur à temps plein pour la lib TLS la plus utilisée au monde).

Conséquence : La sécurité de la supply chain a mis en lumière un problème structurel de **financement** et de **soutenabilité** de l'open source (cf. session 9).

Pour les développeurs

Gestion des dépendances

- Minimiser le nombre
- Vérifier la maintenance
- Verrouiller les versions (lock files)
- Auditer régulièrement

Sécurité

- Activer Dependabot/Snyk
- Mettre à jour rapidement
- Ne pas ignorer les alertes
- Vérifier les nouvelles deps

Évaluer une dépendance

Questions à se poser :

1. **Maintenance** – Dernière release ? Issues traitées ?
2. **Communauté** – Nombre de contributeurs ? Bus factor ?
3. **Sécurité** – CVEs passées ? Temps de réponse ?
4. **Nécessité** – Vraiment utile ? Alternative ?
5. **Licence** – Compatible avec mon projet ?

Outils : OpenSSF Scorecard, Snyk Advisor, Libraries.io

OpenSSF Scorecard

Évalue automatiquement la sécurité d'un projet

Critère	Description
Binary-Artifacts	Pas de binaires dans le repo
Branch-Protection	Protection des branches
Code-Review	Reviews obligatoires
Dangerous-Workflow	Pas de workflows risqués
Maintained	Activité récente
Signed-Releases	Signatures cryptographiques
Vulnerabilities	Pas de vulnérabilités connues

Ref: <https://scorecard.dev/>

OWASP : la communauté qui se prend en main

Open Worldwide Application Security Project – fondation à but non lucratif, créée en 2001, dédiée à la sécurité des applications web.

Modèle

- Communauté ouverte de bénévoles
- Contenu et outils sous licences libres / Creative Commons
- Financée par dons, adhésions, événements
- Présente dans le monde entier (chapters locaux)

Publications phares

- **OWASP Top 10** – les 10 risques web les plus courants (référence mondiale)
- **ASVS** – standard de vérification sécurité
- **Cheat Sheets** – guides pratiques
- **CycloneDX** – format SBOM (cf. Partie 3)

Outils : ZAP (scanner web), Dependency-Check, DefectDojo...

Un **contre-modèle vertueux** : une communauté qui se structure pour produire elle-même les ressources de sécurité que l'industrie seule ne produisait pas.

Pour les organisations

Politique

- Registre de dépendances approuvées
- Process de validation
- SLA de mise à jour
- Formation des développeurs

Outillage

- SCA dans la CI/CD
- SBOM automatique
- Monitoring continu
- Plan de réponse aux incidents

Builds reproductibles

Principe : À partir du même code source, le build produit un résultat **bit-à-bit identique**, quel que soit l'environnement.

Pourquoi c'est important :

- Permet de **vérifier** qu'un binaire distribué correspond bien au code source
- Détecte les altérations malveillantes dans le processus de build (cf. XZ Utils)
- Rend le "Trusting Trust" de Ken Thompson (1984) vérifiable

Projets : <https://reproducible-builds.org/> – Debian, Arch Linux, NixOS y travaillent activement.

Le SLSA Framework

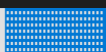
Supply-chain Levels for Software Artifacts

Niveau	Exigences
SLSA 1	Documentation du process de build
SLSA 2	Build hébergé, provenance signée
SLSA 3	Build isolé, provenance non-falsifiable
SLSA 4	Build hermétique, deux parties

Objectif : Garantir l'intégrité de la supply chain.

Partie 6

IA et sécurité open source



Les LLM rebattent les cartes

Les modèles de langage (LLM) transforment la recherche de vulnérabilités et la supply chain – pour le meilleur et pour le pire.

Opportunités

- Détection automatisée de bugs et vulnérabilités
- Génération de patches
- Audit de code à grande échelle
- Aide aux mainteneurs (triage, revue)

Risques nouveaux

- **Slop vulnerability reports** : signalements de masse, souvent faux
- Ingestion non consentie du code par les modèles
- Génération de dépendances "hallucinables" ([*slopsquatting*](#)), variante du (*typosquatting*)
- Agents autonomes exploitant la surface d'attaque

Controverse : Cal.com (avril 2026)

Cal.com (alternative open source à Calendly) annonce une **fermeture partielle** de son code.

Motivations invoquées

- Flot de faux rapports de vulnérabilités générés par LLM – temps mainteneur gaspillé
- Concurrents utilisant les LLM pour cloner la base de code rapidement
- Difficulté à monétiser un OSS totalement ouvert face à ces pressions

Ce que ça illustre

- La fragilité du contrat social de l'open source face à l'IA
- Le lien avec les tensions vues en session 9 (VC, exit, bait-and-switch), i.e. peut-être un prétexte?
- Un précédent inquiétant : d'autres projets pourraient suivre

Source : strix.ai/blog/cal-com-is-closing-its-code-due-to-ai-threats

Les cooldowns de dépendances

Idée simple : attendre quelques jours avant d'installer une nouvelle version d'un paquet.

Pourquoi ça marche : dans la plupart des attaques récentes (Ultralytics, Nx, rspack, Shai Hulud...), la fenêtre entre publication du code malveillant et détection par la communauté est **< 1 semaine**. Un cooldown de 7 à 14 jours aurait bloqué la quasi-totalité de ces attaques.

Adoption massive en 2025-2026 :

- **pnpm, Yarn, Bun, uv** (Python), **npm** – tous ont introduit un paramètre **minimumReleaseAge** / **exclude-newer**
- **Cargo, Go, NuGet** : en cours

Exemple (uv) : `uv add --exclude-newer=7d requests`

À retenir : c'est gratuit, facile, et probablement la mitigation la plus efficace contre les attaques supply chain modernes.

Sources : William Woodruff – blog.yossarian.net ; Cal Paterson – calpaterson.com/deps.html

Bonnes pratiques – l'exemple Astral

Astral (éditeur de `uv` et `ruff`, largement utilisés en Python) a publié son *playbook* sécurité. Quelques principes simples :

Protéger le code et le CI

- Authentification forte (2FA matérielle)
- Droits minimaux par défaut
- Fixer précisément les versions des outils utilisés dans le CI (pas de mise à jour automatique)

Protéger les releases

- Supprimer les mots de passe/tokens long-lived
- Signer chaque release, avec preuve vérifiable d'origine
- Double approbation avant publication

Et côté dépendances : minimiser, appliquer les cooldowns, contribuer aux projets amont dont on dépend, les financer si possible.

URL : astral.sh/blog/open-source-security-at-astral

Résumé

Ce qu'il faut retenir

1. **Supply chain** : 90%+ du code vient de dépendances open source
2. **Incidents** : Log4Shell et XZ Utils montrent les risques
3. **SBOM** : Inventaire obligatoire (CRA, Executive Order) – ****Sécurité + Conformité + Soutenabilité****
4. **CRA** : trois rôles (fabricant, distributeur, *open source steward*), exception OSS non commercial
5. **Réglementation** : RGPD, NIS2, CRA, résistance au FISA et au Cloud Act – l'open source comme levier de souveraineté
6. **IA** : nouveaux risques (slop reports, cal.com) et nouvelles bonnes pratiques (Astral, upload queues)
7. **Outils et pratiques** : Dependabot, Trivy, Trusted Publishing, attestations, minimiser, contribuer upstream

Le paradoxe de la sécurité open source

Avantages

- Code auditable
- Corrections rapides
- Communauté vigilante
- Transparence

Risques

- Mainteneurs non payés
- Surface d'attaque visible
- Confiance automatique
- Dépendances invisibles

■ "Given enough eyeballs, all bugs are shallow" – mais qui regarde ?

Pour aller plus loin

Réglementation

- Texte officiel CRA – digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act
- Guide CRA CNLL / inno³ – code.inno3.eu/ouvert/guide-cra

Standards et frameworks

- **SPDX** – spdx.dev et **CycloneDX** – cyclonedx.org
- **OpenSSF Scorecard** – scorecard.dev
- **SLSA Framework** – slsa.dev
- **Reproducible Builds** – reproducible-builds.org

Communautés et fondations sécurité

- **OWASP** – owasp.org (Top 10, ASVS, ZAP, CycloneDX...)
- **OpenSSF** (Linux Foundation) – openssf.org

Financement de l'OSS

- **Sovereign Tech Fund** (Allemagne) – sovereign.tech
- **Open Source Pledge** – opensourcepledge.com